Benchmarking and Analysis of Common Redstone Computation Techniques

Brandon Esebag, Aaron Hoang, Jonathan Liu, Maria N. Aponte, **Supervisor** : Mohammed El-Hadedy

***Abstract*—Minecraft's redstone system has emerged as an unintentionally powerful medium for introducing players to many fundamental concepts in digital logic and computer architecture. Although redstone circuitry operates within the constraints of the game Minecraft, players routinely construct functional logic gates, memory structures, arithmetic units, and even full general purpose computer systems. This paper benchmarks common redstone computation techniques and analyzes the real engineering challenges that arise when building computational systems in Minecraft. We develop performance metrics tailored to the redstone environment—such as ticks per operation, spatial footprint, and routing cost—and apply them to representative circuits including pulse generators, clocks, memory elements, and more complex systems. Through this analysis, we highlight challenges inherent to redstone computing, such as signal attenuation, clock synchronization, and chunk-loading effects, some of which are present when designed real systems. Finally, we discuss the educational value of redstone as a learning platform, demonstrating how a non-educational game can organically introduce players to complex engineering principles. This work aims to formalize benchmarking approaches for redstone computers while illustrating their relevance for both computational creativity and STEM education.**

***Index Terms*—IEEE, journal, education, benchmarking, Minecraft, redstone**

I. Introduction

THERE are few video games as widely played as Minecraft, and although on the surface it presents itself as a simple children's sandbox game, Minecraft has provided countless people of all ages with the opportunity to learn the basics of computer architecture, knowingly or not. Many members of the online Minecraft community have produced computational components or whole working systems composed of in-game elements. These systems range in complexity from simple working ALUs to entire Minecraft emulations within the game itself.

A. Scope

This paper will focus on benchmarking common computational components and features of Minecraft redstone computers and describing current solutions to common computational and design problems. To benchmark redstone systems, redstone relevant benchmarking metrics will be discussed as standard benchmarking methods cannot be meaningfully applied to redstone systems.

B. An Overview of Minecraft

Minecraft is an open-world sandbox game released on November 18^{th} 2011. The game features a functionally infinite, pseudorandomly generated world composed of cubic-meter blocks. This world can be interacted with through adding or removing blocks, using resources (e.g. blocks or items obtained by breaking blocks) to craft items and tools, and by interacting with mobs (non-player entities).

The game is typically played in "survival mode" which limits player resources and requires the player to obtain food and shelter to survive, however, many players who want to explore the game without such restrictions use the game's "creative mode" which allows unlimited resources, invulnerability to damage, and player flight. This paper will look at the resource costs associated with certain computational techniques; however, it should be noted that many of the players who implement such systems are playing in creative mode, and thus have infinite resources at their disposal.

In Minecraft there are many blocks and ores that can be mined, as suggested by the name "Minecraft", although most of these ores are inspired by real life materials such as iron, copper, diamonds, gold, and lapis lazuli. One of the few ore types in Minecraft that is not modeled off of a real world material is known as redstone. As the name suggests, this ore appears in red ore veins and the material itself is red in color.

C. Minecraft Redstone

When a player mines a redstone ore block with a pickaxe (not enchanted with "silk touch") they can recover around 4-5 "redstone dust", though there are ways of obtaining more redstone dust per ore block using other enchantments. Tool enchantments such as "silk touch" are a game mechanic that falls outside the scope of this paper.

1) Redstone Dust: Redstone dust can be placed on top of blocks to form lines of dust, which can be connected at T-joints or X-joints, and these paths can traverse up the sides of blocks over a distance of 1 block. This means that for two redstone sections, or 'nets' to be isolated, they require 1 block of distance between them.

When redstone dust is placed, it has an "unpowered" state by default. Certain blocks can power redstone dust, making the dark red line of dust glow a bright red. Powered redstone can influence some items in the world, such as extending pistons, opening doors, and similar. If a redstone line terminates at the base of a block, this block will become "powered" though it will not change in appearance.

2) Redstone Schematics: The various available in-game redstone components relevant to "computational redstone" are described below. For visual reference, either when examining in-game screenshots, or for understanding the schematics

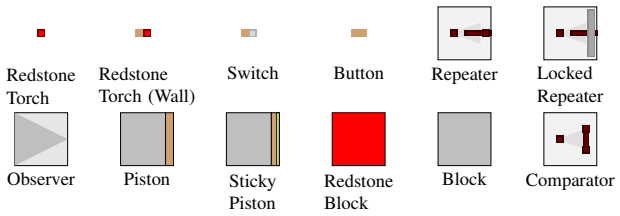

Fig. 1. Symbols for Minecraft Redstone Components

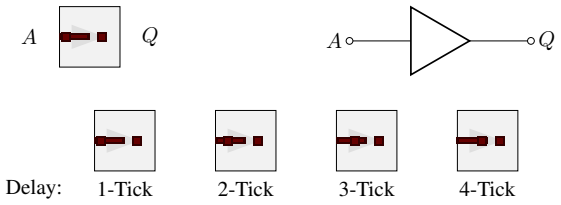

Fig. 2. Redstone Repeater Mechanics

that will be explored in this paper, the appearance of each component has been simplified as shown in Figure 1.

3) Redstone Torch: Using a single wooden stick, and one redstone dust, the player can craft a "redstone torch" which can be placed on the side of most blocks. This torch faintly glows red when placed, and is by default in its "powered state". If the block that anchors the redstone torch is powered, the torch will become "unpowered" and stop glowing. A redstone line that terminates facing a redstone torch can be powered by a redstone torch. This means that in a basic sense, a block with a redstone torch placed on it acts like a 1-bit logic inverter.

4) Redstone Tick: A redstone tick is a unit of time that (barring game lag) takes 0.1 seconds to complete. The power levels of each redstone net position and the application of a redstone signal to a powerable block take 1 redstone tick to complete. The game simulates 20 standard ticks per second which are used to update all other game physics. Unless mentioned, all ticks mentioned will be redstone ticks.

5) Redstone Repeater: Redstone dust can transmit power for a distance of 15 blocks, but due to signal attenuation, it cannot propagate further into the network from the nearest power source. Using three stone blocks, one redstone dust, and two redstone torches, the player can craft an item called a redstone repeater. This device has a triangle on top which is fitting because it acts both as a signal amplifier and as a signal buffer. A redstone repeater always outputs a signal with strength 15 (maximum) so long as the input signal has a strength greater than 0 (fully attenuated).

Redstone repeaters by default impose a 1 tick delay, however this can be adjusted in increments of 1 tick up to 4 ticks, allowing a single repeater to cause 0.4 seconds of delay.

6) Redstone Comparator: Redstone comparators[1] are a redstone component, similar in placement and appearance to a redstone repeater. Comparators have a "back input" and two "side inputs". The default mode of operation outputs the rear input's signal strength only when the rear input is stronger than both side inputs. This behavior can be modeled by Equation 1. A comparator can also be used in "subtraction mode", which will produce outputs in accordance with Equation 2. When in subtraction mode, the front torch of the comparator will glow allowing its operating mode to be visually distinguished.

$$\text{output} = \text{rear} \times [\text{left} \leq \text{rear} \ \wedge \text{right} \leq \text{rear}] \tag{1}$$

$$\text{output} = \max(\text{rear} - \max(\text{left}, \text{right}), 0) \tag{2}$$

7) Other Redstone Components: There are numerous ways the user can power a redstone net, such as pressure plates, buttons (placable on the side of a block), switches (top or side placement), and many more Some of these items are represented in Figure 1. Minecraft also has many sensors such as daylight sensors which can be used to implement logic conditional on the current world state. There are numerous other redstone items, such as redstone blocks (act as a power source), redstone lamps (redstone powered light sources), dispensers (can launch items), powered mine-cart tracks, copper bulbs which can be made to act like T-flip flops (Figure 6) and many more.

D. A Note on Power Consumption

Although Minecraft redstone implements signal attenuation through the use of the aforementioned "redstone power" metric, redstone actuation and signal switching does not actually consume power from any source. Redstone systems are self-powering, and as such the power consumption of a Minecraft computer is not something that can be benchmarked, as power consumption is not a feature of redstone.

E. Research Questions

This research was motivated by the authors' prior experience with redstone-based digital logic, as well as the recognition that Minecraft unexpectedly encourages players to tinker with digital logic circuits without the guise of being an educational tool.

This observation is very important for modeling other forms of educational software, the fact that Minecraft unintentionally encourages its players to grapple with real engineering problems through its implementation of redstone is significant because doing so in-game feels no different than other aspects of gameplay. The game does not instruct the player to use redstone circuitry, it is simply there to be explored by players when they are ready.

Minecraft computers have become significantly more common as internet resources detailing their construction have become more commonplace. As a result of this "explosion" of player creativity, a natural question would be how these computers could be compared to each other given the significant variation and scope of these projects. The relevant research questions resulting from these lines of thinking are as such.

1) How do you benchmark a Minecraft computer?
2) What kinds of real engineering issues arise from Minecraft computing?
3) What unique educational value can Minecraft redstone provide?
4) What makes Minecraft redstone feel so approachable to such a wide audience?

II. Related Work

Ever since redstone was added in July 2010 as a feature to the game, players and the community have experimented by creating all sorts of automated tools including our project's focus. In the first few years of the addition, players first created simple gates and latches. Overtime, players then moved on to more complex digital circuits including full working ALUs, CPUs, and computers. Many websites such as Minecraft Wiki[2] and numerous videos on Youtube have information and demonstrations on how to build and create these circuits.

A. Community Redstone Designs

The first post on the official Minecraft forum page was posted on December 20, 2010 titled "Redstone Logic Gates and FAQs Compendium". This included all logic gates, latches, flip flops, and all digital design elements and circuits. By April 2011, community member "Salaja" posted the first recorded post including a combination of community member designs for ALUs, CPUs, and Computers. To this day, community members have researched and experimented with designs creating more optimized limiting clock speed, propagation delay, and build size. Relevantly the Minecraft community has developed numerous logical implementations of redstone, some simple examples from "mattbatwings"[3] are shown in Figure 3.

B. Minecraft Wiki Contributions

The Minecraft Wiki is a website that extensively documents Minecraft's vast features and mechanics, and in great detail discusses redstone[4] in many of its articles. In the Minecraft Wiki article "Redstone circuits/Memory"[5] for example, a detailed explanation of SR latches, T-Flip Flops, and comparisons of redstone implementations of them. These comparisons implement size and resource cost benchmarking which will be examined further in section III-D below.

C. Minecraft Community Computers

There have been numerous examples of fully functioning computer systems built in Minecraft, though they have only been documented informally. Below is a short list of some notable examples that can be found on Youtube, as most of the people producing these systems are Minecraft Youtubers.

- The Youtuber "mattbatwings" produced a simple computer that can perform mathematical operations and display to a "black and white" screen[6][7].
- The Youtuber "Torb" produced an emulation of Tetris on a new "RGB display" variant[8].
- The Youtuber "sammyuri" produced a "black and white" emulation of Minecraft[9].
- The Youtuber "sammyuri" produced a working pre-trained redstone LLM[10].
- The YouTuber "ModPunchtree" produced a computer called "Iris" which can run Doom (1993), ray-tracing, and a 16-color emulation of Minecraft[11].

A major source of innovation in the Minecraft redstone community has always been display "technology" as the game only provides a limited selection of output devices that these computers can display to. Though this is outside the scope of this paper, the engineering challenges of producing working displays in Minecraft are similarly complex to producing a working computer system.

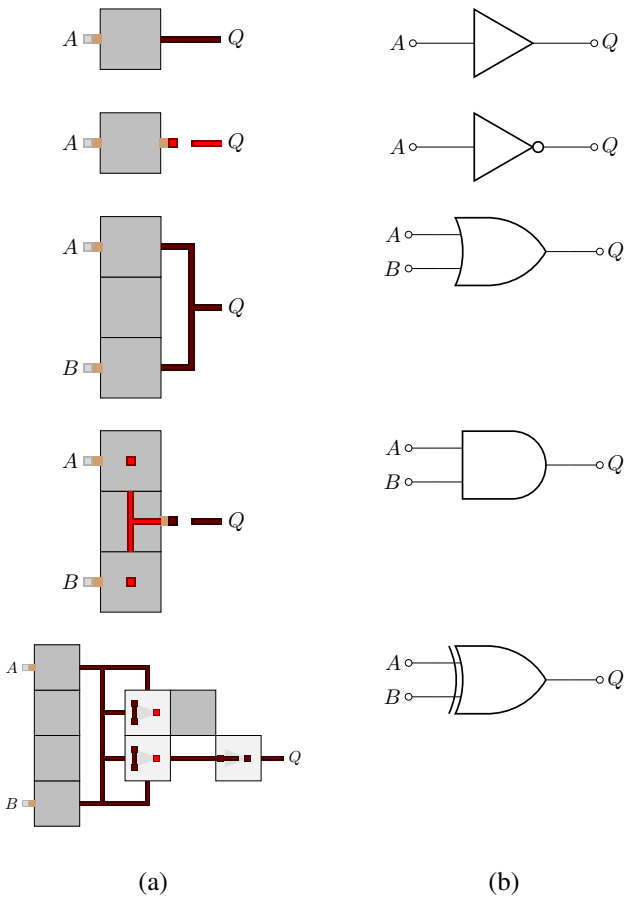

Fig. 3. Common Redstone Logic Gate Topologies (a) and Gate Symbols (b)

III. Benchmarking Methodology

Normal benchmarking methodology is not completely applicable to redstone logic for three important reasons which are as follows:

1) Redstone does not consume power
2) Redstone circuits (at most) operate in the 1-10 Hz range
3) Redstone circuits occupy many cubic meters and require a meter of separation between nets/traces

Metrics like cycles per instruction, data transfer speeds (by bits per Tick) and similar work similarly to normal benchmarking metrics. Because of the relatively slow Tick speed limitations imposed by Minecraft, metrics like "Millions of Instructions per Second" or MIPS become inconvenient, as the fastest device conceivably possible (1 Instruction per Tick) would only achieve 10 instructions per second, or $1 \cdot 10^{-5}$ MIPS. Similarly any analysis of circuit size will need to be adjusted, as even a simple redstone torch inverter (the simplest redstone logic gate) occupies two cubic meters of space.

A. Performance Metrics

1) Ticks per Operation: For devices that perform read, write, or logic computations, performance can simply be measured in Ticks per Operation or TpO. A device having a TpO of 0 implies the device is not amplified and only contains redstone nets, thus practically the lowest TpO a device should have is 1 outside of special cases.

2) Ticks per Computation: For devices that perform more complex tasks such as instruction execution or computation, performance can be measured in Ticks per Computation, or TpC. Similarly to TpO, the lowest TpC a device should normally have is 1, except any computer that can perform parelell processing can achieve lower TpC values. This is not commonly used in Minecraft redstone design because the true bottleneck of computer speed is often the game engine and the computer running Minecraft in the real world, meaning actual Ticks often last much longer than 0.1 seconds.

B. Clocking and Synchronization Considerations

A redstone clock can be created by forming a loop of redstone with two repeaters (they must be separated). One net must be enabled for 1 tick, which will result in the system oscillating between its two stable states. This can be done with larger even numbers of repeaters to make slower clocks if needed. Since redstone suffers signal attenuation, redstone repeaters are often needed to amplify signals over long nets. This means that clock delay will be present in large designs, resulting in H trees and other clock synchronization methods being used. Similarly, worst case analysis needs to be performed on complex circuits to determine the maximum viable clock speed. These are both real life design challenges that are often faced when implementing sequential logic using redstone, and this will effect both TpO and TpC values. We can normalize TpO and TpC in relation to the clock-speed, resulting in Cycles per Operation (CpO) and Cycles per Computation (CpC).

C. Chunk Loading Considerations

In Minecraft, a "chunk" is a 16×16 block region (from the bottom to the top of the world). Only a predetermined number of chunks are loaded at any given time (based on player position), and both redstone and general ticks only trigger updates in loaded chunks. Often techniques are used to keep many chunks loaded, such as abusing in game item properties, but often server commands (in creative worlds) or mods are used to prevent chunk loading issues for large computer projects. the details of chunk loading are outside of the scope of this project, however this is an important consideration when designing a large system.

D. Cost Metrics

1) Material Costs: A computational element can be evaluated based on the recourse costs in producing it, which would simply be a list of all items needed to construct it.

2) Spatial Footprint: Since Minecraft redstone computers often occupy a 3-D volume, the footprint given for any computational element or system will be given in cubic meters instead of square meters.

3) Routing Considerations: As mentioned in Section III-B, signal attenuation often results in using redstone repeaters for spatially long nets. This will impart a delay of at least 1 tick, encouraging efficient routing when designing redstone computers. This is another example of a real engineering problem being disguised as a game-mechanic.

IV. Benchmarking Results

A. Clock and Pulse Circuit Designs

1) Short-Pulse Generation: There are many known redstone pulse and clock generation circuits; however, a common example for each has been chosen for analysis. A common pulse circuit[12] is shown in Figure 4 (2). This circuit can generate a pulse with a minimum width of 2-ticks (the setting shown on the repeater), and a maximum pulse of indefinite length. This is configured by setting the delay of the repeater(s) to the desired pulse length, these can be chained together to make longer delays if needed. This pulse generator can only produce pulses as short or shorter compared to the input pulse, which for a button lasts no longer than 15 ticks, but for a switch can be indefinite. This pulse generator has a delay of 2-ticks (excluding the button), and is triggered on the rising edge of the input signal.

2) Long-Pulse Generation: A circuit that can generate a pulse longer than the input pulse provided[12] is shown in Figure 4 (b). This circuit produces a pulse n ticks longer than the input pulse, where the repeater on the right is configured for an n pulse delay. This works due to the ability of the repeater to power the circuit for n ticks once its source (the input pulse) is disabled. Like the short-pulse generator, this pulse generator has a delay of 2-ticks (excluding the button), and is triggered on the rising edge of the input signal.

3) Clock Generation: A clock can be generated using the circuit shown in Figure 4 (c), where the clock has a period $p = 2 \cdot (1 + n)$ ticks for n-ticks worth of repeater delay configured[12]. The extra tick is present due to the presence of the comparator, as such a 1-tick pulse clock cannot be made using this circuit. Unlike the pulse generation circuits, this circuit needs a continuous input to function, as shown by the use of a switch on the circuit's input. By default this clock will have a duty cycle of 50%, however if a mirrored section is built before output Q, the half-period of that segment will "canceled" out resulting in a lower duty-cycle signal which can be inverted to produce a higher duty-cycle signal.

4) Recourse Costs and Benchmarking: The footprint(s) and costs for these circuits is not as critical as with other components such as RAM modules due to the fact that most computer systems will not have a large number of pulse generator or clock circuits. Though the cost of these components is not critical, it should be noted that all components present are relatively cheap, the most "expensive" material being the 'Nether Quartz' needed for crafting the comparator elements. Though Nether Quartz can only be mined in the Nether (a

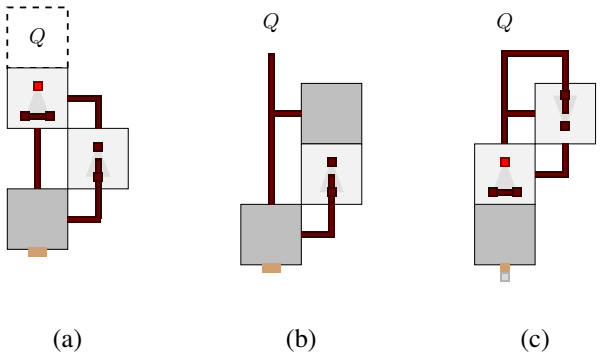

Fig. 4. Short-Pulse (a), Long-Pulse (b) and Clock (c) Circuits

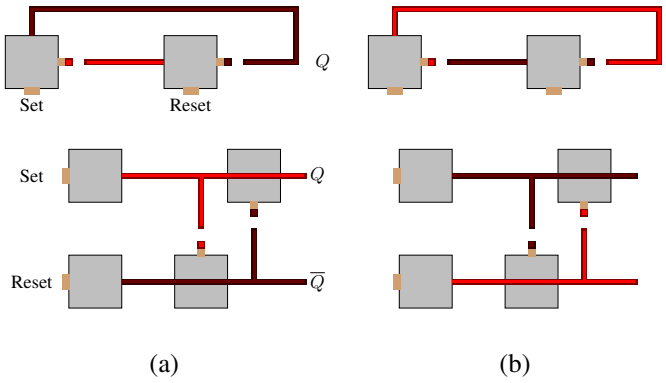

Fig. 5. Common NOR Gate SR Latch Set (a) and Reset (b) States

dangerous, hell-like dimension that can be accessed using a 'Nether Portal'), it is found in large quantities which can usually be found near the portal.

B. 1-Bit Memory Cell Designs

1) Standard 1-bit Memory Circuits: A very simple and common form of redstone memory is the NOR Gate SR Latch[12]. Two examples of this circuit are shown in Figure 5, which when (creatively) compared with Figure 1 is constructed from two connected NOR gates. Several modifications can be made to improve the functionality of this SR Latch, including adding a 'write' signal which can be clocked to prevent state switching between rising edges. This can be done by placing subtraction mode comparators on each input and providing the write signal as a side input for each. Similarly the bottom topology from Figure 5 can be turned into a D-Latch by removing the 'Reset' button block, and supplying said signal from a redstone torch placed on the side of the 'Set' button block. The combination of these two modifications produced a clocked D-Latch, which is another commonly used redstone memory circuit.

It should be noted that the base SR Latch circuit shown has a write time of 1-tick, making it the fastest possible memory circuit available. The addition of a clock synchronization feature (a 'write' signal) adds another tick of delay to the circuit, and the conversion to a D-Latch adds a third tick of delay, substantially slowing down the circuit. This circuit is still better than the others shown below.

2) Creative T-Flip-Flops: Unlike the simple memory circuits shown above, these circuits are notably less space and time efficient, and as such would likely not be used in a computer system. They do demonstrate many interesting redstone properties which are often exploited in more complex way when making computer systems, as such a few of them will be mentioned here:

1) A 'Copper Bulb' (represented in blue-green) is a device that emits light when activated, however unlike the more traditional 'Redstone Lamp' its state flips on the rising edge of an input signal. Due to this interesting behavior, a simple T Flip-Flop can be made from this device using two repeaters and a comparator as shown in Figure 6. This circuit works because the repeater (which could also be redstone dust) powers the lamp forcing a state change. The comparator is needed to 'export' this state onto the output Q, this can be done on the other two sides of the Copper Bulb as well. This is a space efficient and tillable design, the reason for its impracticality if resources are limited would be the necessity for a 'Blaze Rod' to craft the Copper Bulb, as these are difficult to get and have other important uses.
2) The circuit shown in Figure 7 utilizes repeater locking. This occurs when another repeater (which is powered) points into a repeater, which locks its current state. This locking is used to make a working T Flip-Flop, though it requires more vertical clearance than the copper bulb circuit.
3) The circuit shown in Figure 8 utilizes a quirk of Sticky-Piston mechanics. A sticky piston can push a block, and then pull it back (because it is sticky), however in some cases, an input pulse of 1-tick will extend the piston but the piston fail to retrieve the block. This is used to store a bit of memory Q by abusing the update rules regarding locked repeater state (they update first), as the 1-tick pulse generated when the repeater is unlocked will cause this piston behavior to occur. Underneath the piston there is a 1-block section removed (shown as a dashed square) with a redstone torch inside. This torch will power a block above it, powering Q when the piston has its block, but not when the block is lost. The piston will regain the block when it extends again.
4) An 'observer' is a redstone component that observes a state-change behind it, and generates a 1-tick pulse in front of it on the rising-edge of that state-change. Using the same piston mechanic shown in Figure 8, observers can be used to implement a T Flip-Flop as shown in Figure 9. When the button is pressed, the first sticky piston fires, and the observer stuck to it detects this state-change and sends a 1-tick pulse to the second piston. This observer is not lost to the first piston because the button outputs a much longer pulse. The second piston fires pushing a redstone block next to the output Q, which will then be powered by said redstone block. This sequence of steps is illustrated in Figure 9, the reverse of which would be taken when deactivating the bit.

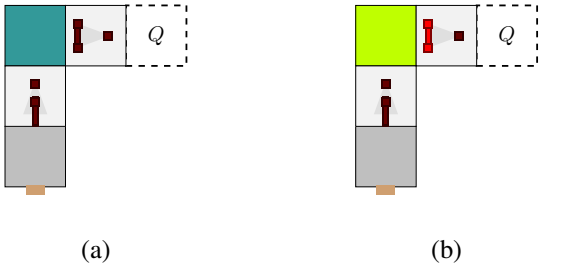

Fig. 6. Copper Lamp based T-Flip Flop Set (a) and Reset (b)

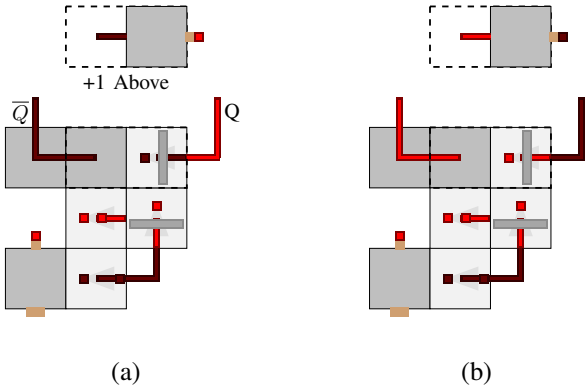

Fig. 7. Repeater based T-Flip-Flop Set Set (a) and Reset (b)

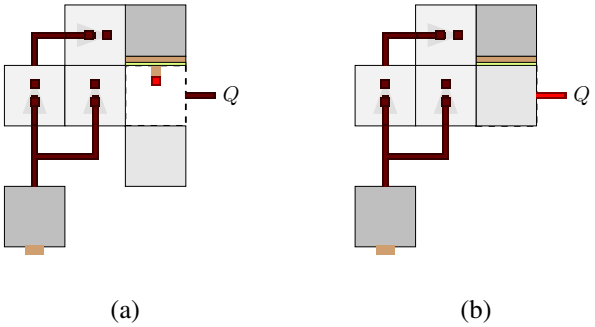

Fig. 8. Sticky Piston based T-Flip-Flop Set (a) and Reset (b)

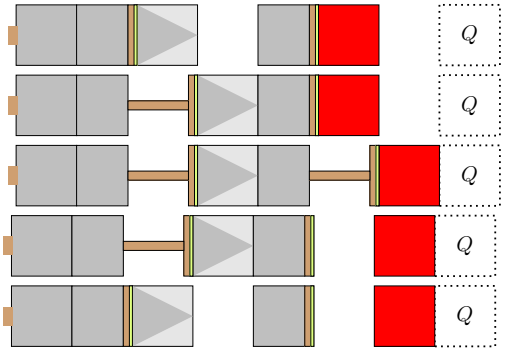

Fig. 9. Observer based T-Flip-Flop Transition Steps

C. Register Bank Designs

Using the building blocks established above, register banks can be designed using any Latch element with a write input signal. A more space efficient but resource expensive form of latch when compared with the double NOR SR Latch design shown in Figure 5 is known as a 'repeater lock Latch'. In this setup the data is stored in a repeater which is locked by another repeater with an input signal $\overline{W}$. On the rising edge of write signal $\overline{W}$ a 2-tick pulse circuit (Figure 4) feeds into a line of torch inverters, typically on a line above the register latches for a horizontal register[13]. This generates the signal $\overline{W}$ at each locking repeater, allowing the data on the input of each latch repeater to be stored before the system locks again.

Often these registers are made vertical for horizontal space efficiency, and the line that carries W is implemented using a "glass tower". In Minecraft redstone can be transmitted through transparent blocks such as glass, as such if glass is placed in two columns with alternating blocks removed, a redstone dust can be placed on top of each remaining block. This structure acts as a vertical wire, and allows for vertical signal transmission to be done in a $1 \times n$ space. This can only be made 15 blocks tall before a repeater is needed, which is the exact height needed to build a vertical 8-bit register (because the signal only needs to travel to the bottom block(s) of the top cell). These vertical registers are often small and can be nicely tiled, allowing for compact register blocks to be built and accessed. If all input data is given to each register input, and each register can generate its own signal W (possibly supplied via a multiplexer or decoder), the writing of different registers can be efficiently handled in a much smaller space. These registers have a write speed of 2-Ticks plus the write time of the D-Latch units, and a read speed of 1-Tick not including the decoding logic required to access each bit.

D. Memory Designs

For the design of program memory[14] or data memory[15] large arrays of registers can be built. Interesting the hierarchy of these registers, and their interconnections are assembled very similarly to how real memory is constructed. This is covered extensively in the cited sources, which discuss information commonly taught in college level digital logic design and computer architecture courses. These constructs are typically large and slow when constructed from redstone due to the signal lag caused by the necessary repeaters.

E. Complex Combinational and Sequential Systems

From the logic gate implementations shown in Figure 3 it can be seen that any possible sequential circuit can be constructed using redstone circuits, and from the discussion above it can be seen that memory circuits can also be constructed using redstone circuits. For larger systems such as Adders[16], Subtracters[17] and ALUs[18], construction using the components outlined is done using typical digital logic design and computer architecture reasoning. There are some cases such as in redstone addition where real life techniques such as carry-look-ahead addition are not commonly implemented because redstone specific techniques that exploit the game logic of redstone circuitry perform better than any real adder

circuit could[16] (in terms of operations required to produce an output). In most cases, computational circuit designs that would function well in the real world typically translate well to Minecraft redstone at this higher level of abstraction, even though the individual components do not function like their real world counterparts.

V. Conclusion

Minecraft's redstone system, though conceived as a simple game mechanic, proves to be a surprisingly rich medium for exploring real concepts in digital logic design and computer architecture. Through the benchmarking framework developed in this paper—centered on metrics like TpO, spatial footprint, and routing cost—we demonstrated that redstone circuits encounter many of the same engineering constraints found in physical systems, including signal attenuation, clock synchronization challenges, and in larger circuits there are even challenges like memory hierarchy design. By analyzing pulse generators, memory cells, register banks, and common sequential and combinational structures, we showed how players organically rediscover engineering trade-offs inside a virtual environment that was never explicitly built to teach them.

These findings reinforce the idea that Minecraft serves as a uniquely accessible educational platform, lowering the barrier to engaging with computational thinking while still exposing users to authentic system-level constraints. The ability for players to construct full ALUs, CPUs, and even entire computers reflects not only the expressive power of redstone but also the creativity of its community. Ultimately, formalizing benchmarking practices for redstone systems not only helps compare community designs but also bridges the gap between playful experimentation and genuine engineering insight. Redstone computing thus stands as an example of how intuitive, open-ended game mechanics can cultivate deep STEM learning in a way that feels natural, exploratory, and inviting to a wide audience.

Acknowledgment

The authors would like to thank the Minecraft game development team, the Minecraft community and the Minecraft Wiki contributors for making the research done for this paper possible. The Minecraft community has generated a wealth of engineering knowledge regarding redstone and related game mechanics that was invaluable to our understanding.

VI. References

References

[1] Minecraft Wiki contributors, "Redstone Comparator," Minecraft Wiki. https://minecraft.fandom.com/wiki/Redstone_Comparator (accessed Dec. 10, 2025).

[2] Minecraft Wiki contributors, "Redstone circuits," Minecraft Wiki. https://minecraft.wiki/w/Redstone_circuits (accessed Dec. 10, 2025).

[3] mattbatwings, "Boolean Algebra & Redstone Logic Gates - LRR #3," YouTube, May 20, 2023. https://www.youtube.com/watch?v=J36WJHaFPGc&list=PL5LiOvrbVo8keeEWRZVaHfprU4zQTCsV4&index=3 (accessed Dec. 10, 2025).

[4] Minecraft Wiki contributors, "Redstone Dust," Minecraft Wiki. https://minecraft.fandom.com/wiki/Redstone_Dust (accessed Dec. 10, 2025).

[5] Minecraft Wiki contributors, "Redstone circuits/Memory," Minecraft Wiki. https://minecraft.fandom.com/wiki/Redstone_circuits/Memory (accessed Dec. 10, 2025).

[6] mattbatwings, "I Made a Working Computer with just Redstone!," YouTube, March 25, 2023 www.youtube.com.https://www.youtube.com/watch?v=CW9N6kGbu2I (accessed Dec. 10, 2025).

[7]] mattbatwings, "I Made a Powerful Redstone Computer!," YouTube, Aug. 17, 2024. https://www.youtube.com/watch?v=3gBZHXqnleU (accessed Dec. 10, 2025).

[8] Torb, "I made RGB Tetris for a Minecraft CPU...," YouTube, Jan. 30, 2023. https://www.youtube.com/watch?v=USH-PME_rls (accessed Dec. 10, 2025).

[9] sammyuri, "I made Minecraft in Minecraft with redstone!," Sep. 6, 2022, www.youtube.com,Sep.06,2022.https://www.youtube.com/watch?v=-BP7DhHTU-I (accessed Dec. 10, 2025).

[10] sammyuri, "I built ChatGPT with Minecraft redstone!," YouTube, Sep. 28, 2025. https://www.youtube.com/watch?v=VaeI9YgE1o8 (accessed Dec. 10, 2025).

[11] ModPunchtree, "DOOM running on REDSTONE COMPUTER - E1M1 in Minecraft!," YouTube, Apr. 06, 2024. https://www.youtube.com/watch?v=_SvLXy74Jr4 (accessed Dec. 10, 2025).

[12] mattbatwings, " Pulses, Clocks, Latches & Flip-flops - LRR #7," YouTube, July 15, 2023. https://www.youtube.com/watch?v=N-edYGFQIjY (accessed Dec. 10, 2025).

[13] mattbatwings, "The Register File - Let's Make a Redstone Computer! #3," YouTube, Sep. 21, 2024. https://www.youtube.com/watch?v=LUQZR8i_t-0&list=PL5LiOvrbVo8nPTtdXAdSmDWzu85zzdgRT&index=4 (accessed Dec. 10, 2025).

[14] mattbatwings, "Instruction Memory - Let's Make a Redstone Computer! #5," YouTube, Oct. 26, 2024. https://www.youtube.com/watch?v=wAwMQp0KNMI (accessed Dec. 10, 2025).

[15] mattbatwings, "Data Memory - Let's Make a Redstone Computer! #9," YouTube, Dec. 29, 2024. https://www.youtube.com/watch?v=KWkGW0lS2-0&list=PL5LiOvrbVo8nPTtdXAdSmDWzu85zzdgRT&index=10 (accessed Dec. 10, 2025).

[16] mattbatwings, "Redstone Binary Addition - LRR #4," YouTube, May 27, 2023. https://www.youtube.com/watch?v=Hl1dHFOl3Zo&list=PL5LiOvrbVo8keeEWRZVaHfprU4zQTCsV4&index=4 (accessed Dec. 10, 2025).

[17] mattbatwings, "Redstone Binary Subtraction - LRR #5," YouTube, Jun 17, 2023. https://www.youtube.com/watch?v=_fZP-r2yhnY&list=PL5LiOvrbVo8keeEWRZVaHfprU4zQTCsV4&index=6 (accessed Dec. 10, 2025).

[18] mattbatwings, "The Arithmetic Logic Unit - Let's Make a Redstone Computer! #2," YouTube, Sep. 7, 2024. https://www.youtube.com/watch?v=ChR7wS94WoY&list=PL5LiOvrbVo8nPTtdXAdSmDWzu85zzdgRT&index=2 (accessed Dec. 10, 2025).